

Zomato Social Watch — Demo Script

for the AI-Native Intern take-home

May 4, 2026

CONTENTS

Zomato Social Watch — Demo Recording Script	
Pre-flight checklist	
SECTION 1 — Open with the problem (0:00 – 0:30)	
SECTION 2 — Live data + classification (0:30 – 1:30)	
SECTION 3 — Triage walkthrough (1:30 – 3:00)	
SECTION 4 — The drain queue (the demo moment) (3:00 – 4:30)	
SECTION 5 — Safety + audit on the dangerous case (4:30 – 5:30)	
SECTION 6 — Close (5:30 – 6:00)	
Brief requirements — visual proof checklist	
Things to keep in mind	
Narrative	
What NOT to do	
Recording mechanics	
Demo risks + mitigations	
The single most important thing	

Zomato Social Watch — Demo Recording Script

Target length: 5–6 minutes. Glance at this during the recording — don't read aloud verbatim.

Pre-flight checklist

Run all of this **before** you hit Record.

```
# 1. Smoke-test connectors
python -m social_watch.actions._smoke --live

# 2. Restart server with reload + auto-reply OFF for the demo
#   (don't accidentally fire 100 templated replies during recording)
AUTO_REPLY_ENABLED=0 uv run python main.py serve --port 8000 --reload
```

Then verify in the browser:

- ☐ `http://localhost:8000` shows “last sync 2m ago” or similar
- ☐ Connector strip on Home shows ≥ 2 channels green
- ☐ Inbox has at least 1 P0 post with a hard tripwire (death claim / legal threat / food safety) — needed for Section 5
- ☐ Browser localStorage has `zsw_operator = "@your-name"` (avoids the prompt during ack)
- ☐ Browser at **1440 px width minimum** (smaller and the inbox columns squeeze)
- ☐ Bookmarks bar hidden, tabs cleaned up
- ☐ Phone on do-not-disturb, quiet room

Test 30 seconds of recording first — watch it back, check audio, check cursor visibility.

SECTION 1 — Open with the problem (0:00 – 0:30)

Screen: Home page. Cursor near the red banner.

Talk track:

Zomato gets thousands of social mentions a day across X and Reddit. About 10% need a public reply within minutes — food safety, delivery failures, founder mentions, legal threats. The team can't keep up: right now we have **186 posts overdue** — meaning more than two hours since they landed and no Zomato reply yet.

The system tells the team three things: *what landed*, *what needs action*, *what the action was* — then either fires that action automatically when it's safe, or lets an operator drain the backlog in one click.

Don't: click anything yet. Let the audience absorb the problem.

SECTION 2 — Live data + classification (0:30 – 1:30)

Screen: Stay on Home. Hover over the connector strip and KPI tiles.

Talk track:

Live data, not mocked. Last sync was 2 minutes ago — the brief asked for ≤ 5 -min refresh. Six connectors total: Reddit and Twitter for read; Slack, Discord, Email, Sheets, Linear for write. Twitter is bidirectional because we reply on it — that's well above the brief's 3-connector minimum.

Every post is classified — urgency, sentiment, audience routing, tripwires. Rules first for cost, Gemini Flash overlay only on the fuzzy axes. The 'Critical now' tile is zero today which is good. 'Overdue' shows 186 — that's our queue. 'Auto-replied' is zero because I have the policy off for this recording.

Action: Brief hover over the volume chart. Pause on a "What's spiking" card to show theme detection. **Don't dwell.**

SECTION 3 — Triage walkthrough (1:30 – 3:00)

Screen: Click **Inbox** in the sidebar.

Talk track:

The inbox sorts by priority. Each row tells me what happened, who said it, how urgent, and what to do. The small **color dot** next to the urgency chip tells me at a glance whether this post is auto-reply-eligible — green means yes, red means a human is needed, grey means Reddit which we keep manual for now.

Action: Hover a **green** dot → tooltip “Auto-reply eligible.” Hover a **red** dot → tooltip “Human required.”

Talk track (continued):

Now the playbook layer. Click any post with a tripwire chip.

Action: Click the **Death claim** chip (or whichever tripwire is in your demo data) → popover opens.

Talk track:

Each hard-tripwire post gets an incident playbook attached. Owner team — Trust & Safety. Acknowledgment SLA — 5 minutes. Required steps: page T&S in #ts-warroom, loop in legal, open a Linear ticket, pull order metadata, mandatory post-incident review within 24 hours. Critically: **auto-reply is BLOCKED** for this category. The corporate handle is one bad classification away from a PR disaster, so the safety guard is at the dispatcher level — not just the UI.

Action: Close the popover with Escape.

SECTION 4 — The drain queue (the demo moment) (3:00 – 4:30)

Screen: Back to Home. Cursor on the red banner.

Talk track:

Now the demo moment. The 186 overdue posts? The classifier already knows which ones the system can safely template-reply to. Click the banner.

Action: Click the red banner → drain modal opens.

Talk track:

It scans the queue, runs every post through ten stacked safety guards: classifier-safe flag, no tripwires, customer-care audience only, two-minute cooling-off so the LLM has time to settle, one-reply-per-author-per-hour throttle. It tells me how many are auto-eligible vs need human review, and groups the human-review pile by playbook so I know what’s pending and why.

Action: Wait for stats to load. Read the breakdown out loud.

Talk track:

Click Drain. Each reply fires through the same code path as a manual reply — Playwright session, templated text, post, verify it landed. Throttled at 1 per second to be polite to Twitter. Live progress bar. Cancel button if I want to stop.

Action: Click **Drain**. Let it process **3–5 posts**, then click **Stop**.

⚠ **Do NOT run the full drain on real users** during recording, even though the safety guards are in place. Cancel after a few.

Talk track:

I cancelled at 5 to keep this demo short, but the audit trail captures everything. Activity log.

Action: Click **Activity** in the sidebar.

Talk track:

Every action ever fired, sorted newest first. Each row reads like a sentence — ‘Replied on X to @user’. The **drain** badge marks the ones I just bulk-fired. The **auto** badge would appear on policy-fired ones if AUTO_REPLY_ENABLED were on. Click any row to open the actual reply tweet on x.com — every fire is verifiable.

SECTION 5 — Safety + audit on the dangerous case (4:30 – 5:30)

Screen: Back to Inbox. Find a death-claim row.

Talk track:

Last thing — what if I try to reply to that death claim? Watch.

Action: Click **Reply** on the death-claim row → modal opens, server returns 423, modal swaps to “Playbook Locked” panel.

Talk track:

The dispatcher returns HTTP 423 Locked. The modal swaps to a playbook lock panel showing the procedure, the owner team, the SLA. To override I'd have to provide an approver name, an authorization reason, and tick the confirm checkbox. Without all three, the send button stays disabled. If I do override — say 'Legal cleared this at 14:32' — the reply fires, but my approver name and reason are written to `action_meta.bypass_approved_by`, and the inbox row gets a permanent purple BYPASS chip. **There is no path to fire a public reply from the bot on a death claim that doesn't leave a trail.**

Action: Close the modal. **Do not bypass.**

Talk track:

Same idea on the SLA side. Acknowledgment countdown ticks down live; if nobody clicks Acknowledge by the deadline, a background sweep re-fires Slack with [ESCALATED]. Twenty-four hours after the action, another sweep auto-creates a Linear post-incident-review ticket with the timeline pre-filled.

SECTION 6 — Close (5:30 – 6:00)

Screen: Back to Home or any clean view.

Talk track:

Quick recap on the brief: live data, sub-five-minute refresh, ≥ 3 connectors with read-write — we have six, with one bidirectional. ≥ 1 real action per escalated post — five firing channels plus an automated policy and an operator drain. Working dashboard with full action triggers, audit trail, and post-incident review.

Three things I'd build with more time:

1. **Reddit auto-reply.** The drain logic generalizes once we have OAuth credentials.
2. **LLM-confidence threshold** on auto-fires for high-follower accounts.
3. **Per-assignee Linear routing** that learns which on-call closes which ticket type fastest.

Thanks for watching.

Brief requirements — visual proof checklist

Make sure each of these is visible on screen at least once. The interviewer scores against these.

Brief checkbox	Visual proof
Live data, real not mock	Connector strip green + “last sync 2m ago” timestamp
Refresh ≤ 5 minutes	Same timestamp; mention out loud
Classify every post	Urgency chips + tier badges on inbox rows
Score by priority	Inbox sorted by Priority + colored KPI tiles
≥ 1 real action per escalated post	Activity log with timestamps + clickable reply URLs
≥ 3 live connectors with read/write	Connector strip showing 6; mention out loud
Per-post action triggers in dashboard	Open the $\cdots$ menu once; close it
Top 5 escalations from live feed	Walk through 2–3 P0 rows in the inbox
Working dashboard at localhost	The whole demo

Things to keep in mind

NARRATIVE

1. **Tell one story end-to-end.** The script follows a single arc: 186 overdue $\rightarrow$ triage $\rightarrow$ safe drain $\rightarrow$ safety lock on the dangerous case. Don’t break the arc to show Briefing or Operations or Discovery — those are nice-to-haves; cut them ruthlessly.
2. **Lead with the user, not the tech.** Open with “the team is drowning” — not “I built FastAPI + SQLite.”
3. **End on a strength.** The closing is what they remember. The “what I’d build next” framing shows roadmap thinking and sounds far more confident than feature exhaustion.

WHAT NOT TO DO

- **Don’t read the script verbatim.** Talk in your own words. Reading is audible.
- **Don’t apologize for unconfigured connectors.** The Setup pills + inline env-var hints are a *feature*. Frame: “the moment a credential lands in `.env`, that channel turns green and the menu item enables — no code change required.”
- **Don’t gloss over what’s not done.** Pretending Reddit auto-reply works when it doesn’t is the fastest way to lose credibility. Acknowledge gaps in the closing.
- **Don’t run the full drain on real users.** Cancel after 3–5. Even with all safety guards, a recording is the wrong place to stress-test.

- **Don't show every page.** Skip Briefing, Operations, Discovery in the main demo. Mention them once if you have spare seconds.
- **Don't speed-talk.** Slightly slower than normal. Pauses between sections give the audience time to process.

RECORDING MECHANICS

- **One take is fine.** Stumbles humanize. “Let me click that again” is OK. Restarting the whole video is not.
- **Trim aggressively in post.** A 6-minute script becomes a tight 5-minute video after cutting “ums” and dead air.
- **Use a hardware mic** if you have one. Built-in laptop mics sound tinny.
- **Cursor visibility:** macOS → enable “Shake mouse pointer to locate.” Some recorders also have “highlight clicks” — turn it on.
- **Hide your bookmarks bar** for a cleaner browser frame.

Demo risks + mitigations

Risk	Mitigation
Twitter cookies expire mid-demo	Re-extract before recording; have a screenshot of a successful reply ready as fallback
Drain throws on a specific post	Cancel and skip — say “let me cancel for time” rather than getting stuck debugging
Gemini free-tier rate limit hits	Disable LLM overlay before recording (<code>unset GEMINI_API_KEY</code>); rules-only classification still works
Background sync runs and the queue count changes	Stop the bg loop, restart with <code>--no-watch</code> ; or briefly acknowledge “the system is still pulling”
You forget to mention something	The recap section at the end catches almost any miss — use it to backfill
Server crashes mid-recording	Have screenshots of every section ready as static fallback

The single most important thing

Show, don't tell. Every spoken claim should be backed by a click within 3 seconds.

- "Six connectors" → screen shows six green dots.
- "Audit trail" → screen shows the Activity log row.
- "Refusal on death claim" → screen shows the lock modal.

If you say something and the screen doesn't prove it, the interviewer assumes it doesn't exist.